

Table of Contents

1. Shared Links	1
1.1. Concepts	1
1.2. ShareLink Model	1
1.3. Link Lifecycle	2
1.4. Repository Operations	2
1.5. Access Logging	2
1.6. Link Signing with HMAC	3
1.7. Security Considerations	4
1.8. See Also	4

1. Shared Links

Quantum provides a secure link-sharing mechanism for granting time-bound, usage-limited access to resources (typically cloud-stored files). Share links are tracked with full audit logging and can be revoked at any time.

1.1. Concepts

Concept	Description
ShareLink	A persistent entity representing a shareable link to cloud storage. Contains the S3 coordinates (bucket, region, object key), expiration rules, and usage tracking.
SharedLink	A lightweight DTO returned to callers after link creation. Contains the shareable URL, token, expiration metadata, and audience information.
ShareAccessLog	An audit trail entry recording each access attempt against a share link, including requester identity, IP, user agent, and outcome.
LinkSigner	A cryptographic utility that signs and verifies link tokens using HMAC-SHA256 to prevent tampering and enforce time-bound validity.

1.2. ShareLink Model

The **ShareLink** entity is stored in MongoDB and tracks the full lifecycle of a shared link.

Field	Type	Description
publicId	String	External-facing identifier used in URLs. Not the MongoDB _id .
bucket	String	S3 bucket name
region	String	AWS region of the bucket
objectKey	String	S3 object key (path within the bucket)
status	Enum	ACTIVE or REVOKED
expiresAt	Date	Optional expiration timestamp. Null means no expiration.
maxUses	Integer	Optional maximum number of accesses. Null or 0 means unlimited.
usedCount	int	Number of times the link has been consumed (starts at 0)
allowAnonymous	boolean	Whether unauthenticated users can access the link
contentDisposition	String	Optional content disposition header (e.g., attachment; filename="report.pdf")
createdBy	String	User ID of the link creator
lastAccessedAt	Date	Timestamp of most recent access
createdAt / lastUpdatedAt	Date	Audit timestamps

1.3. Link Lifecycle

```
Create ShareLink (status=ACTIVE)
  |
  v
Generate signed URL with LinkSigner
  |
  v
Return SharedLink DTO to caller
  |
  v
Access: ShareLinkRepo.tryConsumeOneUse()
  +---> Verify ACTIVE status
  +---> Check expiresAt not passed
  +---> Check usedCount < maxUses
  +---> Atomic increment usedCount
  +---> Log ShareAccessLog entry
  |
  v
Revoke: ShareLinkRepo.revokeByPublicId()
  +---> Set status=REVOKED
  +---> Further access attempts are rejected
```

1.4. Repository Operations

The `ShareLinkRepo` provides three key operations beyond standard CRUD:

- `findByPublicId(String publicId)` — Look up a link by its public identifier
- `revokeByPublicId(String publicId)` — Mark a link as `REVOKED` and update the timestamp
- `tryConsumeOneUse(ShareLink link)` — Atomically consume one use of the link. Returns `true` only if the link is active, not expired, and within its usage limit. Uses a Morphia session to prevent race conditions.

1.5. Access Logging

Every access attempt (successful or not) should be recorded in `ShareAccessLog`:

Field	Type	Description
<code>linkId</code>	<code>ObjectId</code>	Reference to the <code>ShareLink</code> document
<code>publicId</code>	<code>String</code>	The public identifier of the link
<code>ts</code>	<code>Date</code>	Access timestamp
<code>requesterIdentity</code>	<code>String</code>	Subject or user ID of the requester (or "anonymous")
<code>ip</code>	<code>String</code>	Source IP address
<code>userAgent</code>	<code>String</code>	Request user agent string

Field	Type	Description
<code>outcome</code>	String	Result of the access attempt (e.g., "OK", "EXPIRED", "REVOKED", "MAX_USES_EXCEEDED")
<code>message</code>	String	Optional detail message

1.6. Link Signing with HMAC

The `LinkSigner` utility provides cryptographic signing and verification of share link tokens using HMAC-SHA256.

Configuration

```
# Secret key for HMAC signing (required for link signing to be active)
share.link.signing.secret=${SHARE_LINK_SECRET}
```

If the secret is not configured, `LinkSigner.isEnabled()` returns `false` and signing/verification is skipped.

Signing

```
@Inject
LinkSigner linkSigner;

if (linkSigner.isEnabled()) {
    long now = Instant.now().getEpochSecond();
    String signature = linkSigner.sign(shareLink.getPublicId(), now);
    // Include signature and timestamp in the shareable URL
}
```

Verification

```
// Verify with default 24-hour clock skew tolerance
boolean valid = linkSigner.verify(publicId, timestamp, signature);

// Verify with custom skew tolerance (e.g., 1 hour)
boolean valid = linkSigner.verify(publicId, timestamp, signature, 3600);
```

The verifier checks:

1. The HMAC signature matches for the given `publicId + timestamp`
2. The timestamp is within the allowed clock skew window (default: 24 hours)
3. Comparison uses constant-time equality to prevent timing attacks

1.7. Security Considerations

- Always configure `share.link.signing.secret` in production with a strong random value
- Use `expiresAt` and `maxUses` to limit link exposure
- Set `allowAnonymous=false` unless guest access is explicitly required
- Monitor `ShareAccessLog` for anomalous access patterns
- Revoke links immediately when they are no longer needed

1.8. See Also

- [Permissions and Authorization](#)
- [Authentication and Authorization](#)
- [Configuration Reference](#)